

Agent 决策机制与治理依赖 — 讨论整理

三个递进的问题:agent 靠什么"决策"?它依赖 Palantir 的**本体论**吗?依赖与否的代价是什么?结论先行:**认知上零依赖,制度上需要一个采纳层——但那层不必是 Palantir 的。**

承接《受治理Action层设计讨论》;蓝=Palantir 工程,橙=《决策原生方法论》制度观,青=综合落地。机制描述经案例 06(Agno×Semantica)实证参照。



深色

目 录 [1 机制解剖](#) [2 依赖之辨:认知 vs 制度](#) [3 采纳层的选择](#) [4 优缺点对照](#) [5 综合切分:判断自由,采纳受治](#)

1 机制解剖:agent 靠什么"决策"

先纠正前提:agent 自己并不"做决策",它只产生"判断"。机制分三层——每层的确定性、责任归属完全不同。

L1

核心

LLM 核心 —— 概率性判断

下一 token 预测:基于训练分布的条件生成。不是深思熟虑,是统计模式匹配;温度采样 → **不可复现**。这一层是 agent 的"直觉",本体论管不着它。

L2

循环

Agent 循环 —— 感知 → 推理 → 行动 → 观察 → 迭代

每一轮"决策" = LLM 在上下文(提示 + 记忆 + 工具返回)约束下选择下一个动作。工具返回把世界喂给它;循环给它试错空间。

L3

委托

工具委托 —— 判断可被"外包"给确定性引擎

agent 调用规则引擎/推理机(Rete / Datalog / 查询)时,判断质量由被委托引擎决定——**可以是确定性的**。△ PDF 的 Skill:"Skill 不承担决策责任,核心是可替换性与可审计性,而非智能程度。"

关键认知:这套机制完全在本体论之外运行。案例 06 的裸 Agno agent 不需要任何本体就能说"我认为该批准"——那只是一次判断,不构成对任何人的约束。

2 依赖之辨:认知依赖 vs 制度依赖

"依赖 Palantir 吗"其实是两个被混在一起的问题——答案相反。

问题	答案	说明
agent 能不能判断? 认知问题	能,零依赖	LLM 本身就是判断机器;本体论不参与它的认知过程
agent 的判断能不能"算数"? 制度问题	需要一层东西 但不必是 Palantir 的	判断 → 决策需要"采纳层"(校验/审批/写回/入账);这层有多种实现可替换

依赖的是"采纳层的存在",不是"Palantir 的采纳层"。Palantir Ontology 只是采纳层的一个专有实现——把"需要制度"误当成"需要某家产品",是这个领域最常见的混淆。

实证参照:案例 06 的 agent 在 Semantica 上跑,全程没碰 Palantir——agent 产生判断, `record_decision` 把它记为可审计对象。开源采纳层同样成立。

3 采纳层的选择:三个层级

从裸 agent 到受治理行动层,是光谱而非二值——每一级买入不同的东西。

层级 0 裸 agent(LLM + 工具) 有判断,无治理。写日志即效果,判断静默生效。	层级 1 agent + 任意共享结构 (图 / 库 / 向量存储)有共享状态,多 agent 可读同一份; 但写回仍不受治。	层级 2 agent + 受治理行动层 判断必须过类型化校验 / 审批 / 写回 / 入账才生效。 判断在此成为决策。
层级 2 的实现选项	性质	代价
Palantir Ontology / AIP	专有、重、最完整(Objects/Actions/审批/权限)	平台锁定、高门槛
Semantica + PDF 制度观 ★	开源、轻;决策节点/审批链/溯源已有,缺阻断式引擎	治理骨架需自建(本 fork 的方向)
自建审批中间件	最轻、最贴身	治理债自己背(校验/审计/版本全要造)

4 优缺点对照:依赖 vs 不依赖

"依赖"= 走受治理采纳层;"不依赖"= 裸 agent 直接生效。各有真实的使用域。

+ 依赖(有受治理采纳层)

- 判断可被采纳为决策:类型化校验、审批闸、写回入账——组织真正受约束
- 共享世界模型:多 agent / 人读同一状态,无复制同步、无状态漂移
- 可审计可追责:谁、何时、依据什么、谁批的——可回放(PDF 第一性约束)
- 接地:读结构化事实而非自由文本,幻觉面收窄
- 责任显式:HITL 闸口钉死"谁担责",不在自动化中隐性漂移
- 生效路径确定:判断即使非确定,采纳过程可复现

- 代价

- 前置建模成本高:先定义对象/Action 类型才能跑;冷启动贵
- 刚性:schema 变更走治理,演进慢;快变领域吃亏
- 延迟与摩擦:低风险决定配两步审批 = 过度设计
- 单写通道可能成吞吐瓶颈
- 锁定风险:Palantir 尤甚;涌现现象被类型系统挤压

+ 不依赖(裸 agent / 仅共享存储)

- 零建模成本,当天上线;探索期友好
- 完全灵活:无 schema 束缚,领域怎么变就怎么来
- 无锁定、无平台税
- 低风险高频小决策的合理选择

- 代价

- 判断静默生效:agent 写了就算,没有"采纳"环节
- 无共享事实源:多 agent 各说各话
- 审计只有日志:不可回放、不可追责;监管场景直接出局
- 幻觉无接地约束
- 责任弥散:"是 agent 干的"不是答案

5 综合切分:判断自由,采纳受治

这不是二选一——《决策原生方法论》的判断/采纳三分,恰好就是这道题的标准答案。

◆ 结论:认知开放 + 制度收敛

判断层保持自由(不依赖任何本体论)——agent 用什么机制想都行:LLM 直觉、委托规则引擎、问人,机制可插拔(≠ Skill 可替换)。

采纳层必须受治(依赖一个治理结构)——但结构可替换:Palantir 是重专有实现,Semantica + PDF 是轻开源实现,自建是最小实现。

依赖的价值不在让 agent 想得更好,而在让它的想法算数——是效力问题,不是智力问题。

按场景选层级

场景	推荐层级	理由
探索 / 研究 / 内部工具	层级 0-1	快、灵活;错误的代价可承受
多 agent 协作但低风险	层级 1	要共享状态,还不需要重治理
受监管 / 高风险 / 不可逆 (放款、用药、豁免)	层级 2	可追责与可回放是 准入条件 ,不是优化项
两者混合(常态)	分级路由	风险阈值分流:低风险自动放行,高风险进审批 ≡ Action 风险分级

对 fork 的意义

Palantir = "认知开放 + 制度收敛"的**重专有实现**;Semantica + PDF = 同一切分的**轻开源实现**。这个 fork 的定位由此明确:**不依赖 Palantir,但要实现 Palantir 解决的那个问题**——判断自由地产生,决策受治地生效。具体缺口仍是那份 Action 引擎原型(校验前置 / 风险分级路由 / 不批不写回 / 全链入账)。

文档说明 — 整理自"agent 决策机制 / 是否依赖 Palantir 本体论 / 优缺点"的讨论,承接《受治理Action层设计讨论.html》。
机制描述参照案例 06(Agno × Semantica,判断层零本体依赖的实证);制度概念出自 [book/《决策原生方法论》.pdf](#) (判断/采纳三分、Skill、强制 HITL);Palantir 部分基于公开文档。
关联: [examples/case/06_agno_two_agents/](#) · [book/受治理Action层设计讨论.html](#) · [book/Agno双Agent决策序列图.html](#)